# Pengembangan Sistem Manajemen Database Terintegrasi untuk Berkat Dinasti

## dalam Mendukung Digitalisasi UMKM di Indonesia

## PENCIPTA

Aditya Widyatamaka Tri Utomo (250535623587)
Chasbiah Azzahra (250535621731)
Dzauq Bachrul 'Ulum (250535621835)
Elshifa Viorissa (250535621859)
Dr. Ir. Triyanna Widiyaningtyas, M.T.
Ir. Ilham Ari Elbaith Zaeni, S.T., M.T., Ph.D.
Harits Ar Rosyid, S.T., M.T., Ph.D.
Wahyu Muhammad Noble Shafa

## I. Judul

Pengembangan Sistem Manajemen Database Terintegrasi untuk Berkat Dinasti dalam Mendukung Digitalisasi UMKM di Indonesia

## II. Latar Belakang

Usaha Mikro, Kecil, dan Menengah (UMKM) merupakan tulang punggung perekonomian Indonesia. Berdasarkan data Kementerian Koperasi dan UKM, lebih dari 99% pelaku usaha di Indonesia berasal dari sektor UMKM, dengan kontribusi yang signifikan terhadap Produk Domestik Bruto (PDB) serta penyerapan tenaga kerja. Namun, di tengah pesatnya perkembangan teknologi informasi, sebagian besar UMKM masih menghadapi berbagai tantangan dalam melakukan transformasi digital, khususnya dalam hal pengelolaan data dan penerapan sistem informasi yang terintegrasi.

Digitalisasi menjadi kebutuhan yang tidak dapat dihindari bagi UMKM agar mampu bersaing, beradaptasi, dan berkembang di era ekonomi digital. Salah satu aspek penting dalam digitalisasi adalah pengelolaan data yang efisien, akurat, dan terstruktur. Penggunaan sistem manajemen database terintegrasi dapat menjadi solusi strategis dalam meningkatkan efisiensi operasional, meminimalisir kesalahan pencatatan, serta mendukung pengambilan keputusan berbasis data.

Melalui tugas akhir mata kuliah Basis Data pada Program Studi Teknik Informatika, mahasiswa diberikan kesempatan untuk mengimplementasikan konsep-konsep yang telah dipelajari ke dalam dunia nyata melalui kerja sama dengan mitra usaha. Dalam hal ini, tim mahasiswa menjalin kolaborasi dengan Berkat Dinasti, salah satu pelaku UMKM di bidang kuliner yang bergerak dalam penyediaan makanan dan minuman. Usaha ini masih menghadapi kendala dalam pengelolaan data transaksi, pencatatan stok, serta penyusunan laporan yang belum terintegrasi secara sistematis.

Sebagai bentuk kontribusi dalam mendukung transformasi digital UMKM, dikembangkanlah sebuah sistem manajemen database terintegrasi yang dirancang sesuai dengan kebutuhan operasional Berkat Dinasti. Sistem ini mencakup pengelolaan data produk, transaksi penjualan, data pelanggan, manajemen karyawan,

serta pengelolaan stok bahan baku. Dengan adanya sistem ini, diharapkan proses operasional menjadi lebih efektif, efisien, dan terorganisir dengan baik.

Dokumentasi sistem ini disusun dalam bentuk *manual book* berjudul *"Pengembangan Sistem Manajemen Database Terintegrasi untuk Berkat Dinasti dalam Mendukung Digitalisasi UMKM di Indonesia"*. Manual ini difokuskan pada perancangan dan implementasi basis data sebagai inti dari sistem yang dibangun, bukan pada pengembangan aplikasi secara keseluruhan. Hal ini sejalan dengan tujuan utama sebagai bagian dari tugas akhir yang akan diajukan untuk memperoleh Hak Kekayaan Intelektual (HKI).

Basis data yang dikembangkan dirancang dengan struktur tabel yang saling berelasi dan mengikuti prinsip normalisasi untuk menjaga integritas data serta meningkatkan kinerja sistem. Tabel-tabel tersebut mencakup data utama yang dibutuhkan dalam operasional usaha, seperti data pengguna, kategori, produk, varian_produk, zona, pelanggan, pesanan, detail pesanan, pembayaran, pengiriman, tracking_log, pengeluaran, transaksi_kas, karyawan, serta setting . Dengan struktur yang terintegrasi, sistem ini diharapkan mampu mendukung skalabilitas dan pengembangan di masa mendatang.

Melalui penyusunan manual book ini, diharapkan pihak Berkat Dinasti sebagai pengguna sistem dapat memahami alur dan struktur basis data yang digunakan. Selain itu, proyek ini juga menjadi salah satu bentuk kontribusi mahasiswa dalam mendukung digitalisasi UMKM di Indonesia melalui penerapan teknologi yang aplikatif dan berkelanjutan. Dengan demikian, kegiatan ini tidak hanya memberikan manfaat bagi mitra usaha, tetapi juga memperkuat sinergi antara dunia pendidikan dan dunia industri dalam mewujudkan transformasi digital nasional.

## III. Tujuan

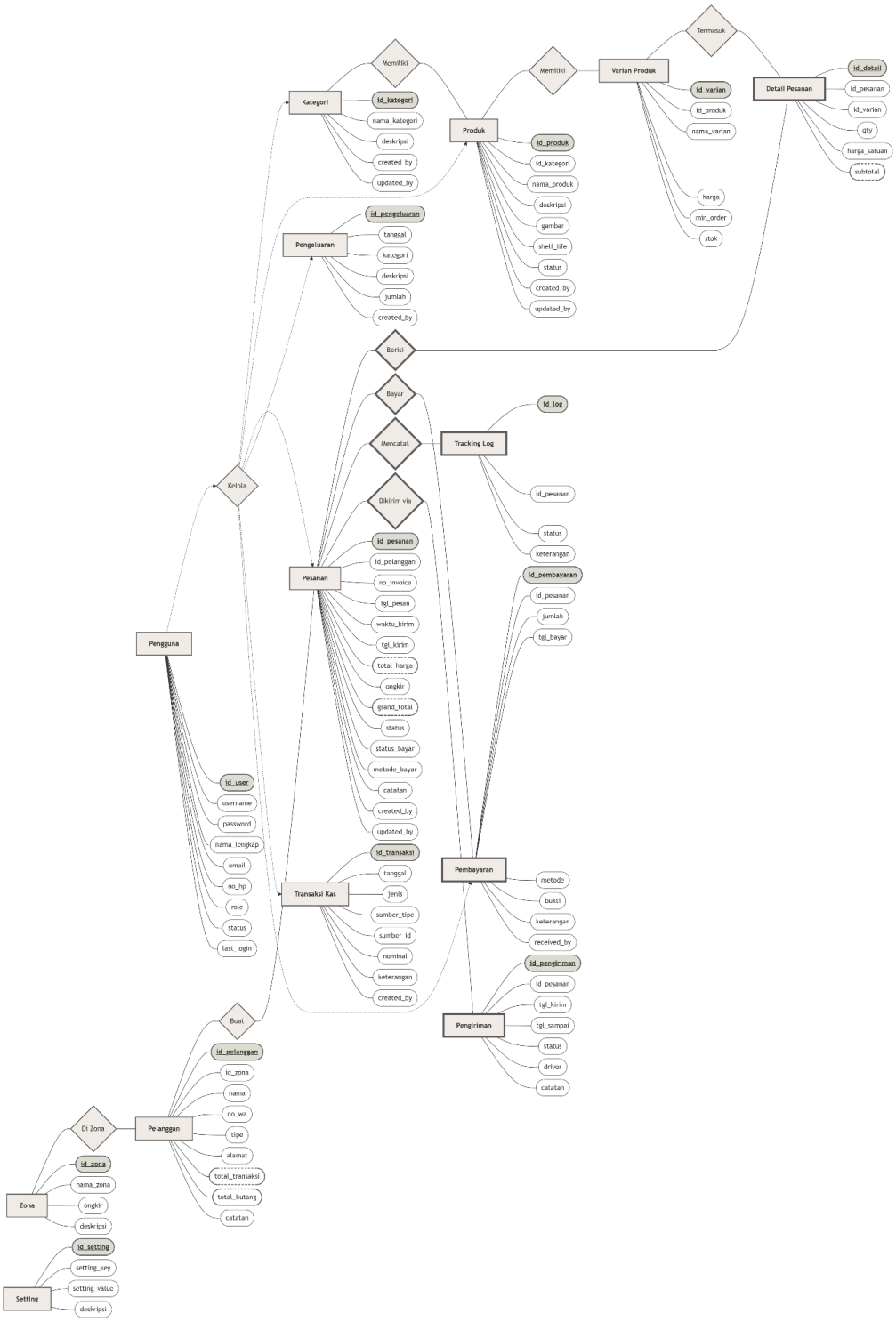
Penyusunan *manual book* ini bertujuan untuk:

a) Mengimplementasikan konsep basis data relasional dalam bentuk sistem manajemen database terintegrasi pada Berkat Dinasti.
b) Merancang dan membangun struktur basis data yang sesuai dengan kebutuhan operasional usaha.

c) Meningkatkan efisiensi dan efektivitas dalam pengelolaan data melalui sistem yang terstruktur dan terintegrasi.

d) Membangun fondasi sistem informasi yang fleksibel dan dapat dikembangkan di masa depan.

## IV. Rancangan Database yang Dikembangkan




Gambar 1.1 Rancangan Basis Data

## V. Source Code

### a. Database berkat_dinasti_db

```
CREATE DATABASE IF NOT EXISTS berkat_dinasti_db
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

USE berkat_dinasti_db;
```

Bagian ini menetapkan basis data utama aplikasi. Penggunaan `utf8mb4` dan `utf8mb4_unicode_ci` memastikan penyimpanan data teks berjalan konsisten dan mendukung karakter lengkap. Perintah `USE` mengarahkan seluruh objek berikutnya ke basis data yang sama.

### b. Tabel pengguna

```
CREATE TABLE pengguna (
    id_user INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL COMMENT 'Hashed password',
    nama_lengkap VARCHAR(150) NOT NULL,
    email VARCHAR(100),
    no_hp VARCHAR(20),
    role ENUM('admin', 'kasir', 'driver') DEFAULT 'kasir',
    status ENUM('aktif', 'nonaktif') DEFAULT 'aktif',
    last_login DATETIME,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Tabel `pengguna` menyimpan akun yang mengoperasikan sistem. Kolom `username` bersifat unik sebagai identitas login, sedangkan `password` menyimpan hash kata sandi. Kolom `role` membedakan hak akses, dan `status` menunjukkan kondisi akun aktif atau nonaktif.

### c. Tabel kategori

```
CREATE TABLE kategori (
    id_kategori INT PRIMARY KEY AUTO_INCREMENT,
    nama_kategori VARCHAR(100) NOT NULL,
    deskripsi TEXT,
    created_by INT,
    updated_by INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Tabel `kategori` digunakan untuk mengelompokkan produk, misalnya kategori roti hajatan dan roti manis. Kolom `created_by` dan

`updated_by` membantu pencatatan aktivitas pengelolaan data agar tetap dapat ditelusuri.

**d. Tabel Produk**

```
CREATE TABLE produk (
    id_produk INT PRIMARY KEY AUTO_INCREMENT,
    id_kategori INT NOT NULL,
    nama_produk VARCHAR(150) NOT NULL,
    deskripsi TEXT,
    gambar VARCHAR(255),
    shelf_life INT DEFAULT 7 COMMENT 'Masa kedaluwarsa dalam hari',
    status ENUM('tersedia', 'tidak_tersedia') DEFAULT 'tersedia',
    created_by INT,
    updated_by INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_produk_kategori
        FOREIGN KEY (id_kategori) REFERENCES kategori(id_kategori)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Tabel `produk` menyimpan data utama barang yang dijual. Setiap produk terhubung dengan satu kategori melalui `id_kategori`. Kolom `shelf_life` mencatat masa simpan produk, sedangkan `status` digunakan untuk menandai ketersediaan.

**e. Tabel varian_produk**

```
CREATE TABLE varian_produk (
    id_varian INT PRIMARY KEY AUTO_INCREMENT,
    id_produk INT NOT NULL,
    nama_varian VARCHAR(100) NOT NULL,
    harga DECIMAL(10,2) NOT NULL,
    min_order INT DEFAULT 1,
    stok INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_varian_produk
        FOREIGN KEY (id_produk) REFERENCES produk(id_produk)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Tabel `varian_produk` menyimpan variasi produk, seperti bentuk kemasan atau ukuran jual. Satu produk dapat memiliki beberapa varian dengan harga berbeda. Kolom `min_order` dipakai untuk menerapkan batas pembelian minimal.

**f. Tabel zona**

```sql
CREATE TABLE zona (
    id_zona INT PRIMARY KEY AUTO_INCREMENT,
    nama_zona VARCHAR(100) NOT NULL,
    deskripsi TEXT,
    ongkir DECIMAL(10,2) DEFAULT 0.00,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Tabel `zona` menyimpan wilayah pengiriman beserta ongkos kirim standar. Data ini menjadi dasar perhitungan ongkir pada saat pesanan dibuat.

**g. Tabel Pelanggan**

```sql
CREATE TABLE pelanggan (
    id_pelanggan INT PRIMARY KEY AUTO_INCREMENT,
    id_zona INT,
    nama VARCHAR(150) NOT NULL,
    alamat TEXT NOT NULL,
    no_wa VARCHAR(20) NOT NULL,
    tipe ENUM('retail', 'reseller') DEFAULT 'retail',
    total_transaksi DECIMAL(15,2) DEFAULT 0.00,
    total_hutang DECIMAL(15,2) DEFAULT 0.00,
    catatan TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_pelanggan_zona
        FOREIGN KEY (id_zona) REFERENCES zona(id_zona)
        ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Tabel `pelanggan` menyimpan data pelanggan yang melakukan pembelian. Kolom `tipe` membedakan pelanggan retail dan reseller. Kolom `total_transaksi` dan `total_hutang` digunakan untuk mencatat akumulasi belanja dan piutang pelanggan.

### h. Tabel pesanan

```
CREATE TABLE pesanan (
    id_pesanan INT PRIMARY KEY AUTO_INCREMENT,
    id_pelanggan INT NOT NULL,
    no_invoice VARCHAR(50) NOT NULL UNIQUE,
    tgl_pesan DATE NOT NULL,
    tgl_kirim DATE,
    waktu_kirim TIME,
    total_harga DECIMAL(15,2) NOT NULL DEFAULT 0.00,
    ongkir DECIMAL(10,2) DEFAULT 0.00,
    grand_total DECIMAL(15,2) NOT NULL DEFAULT 0.00,
    status ENUM('pending', 'proses', 'kirim', 'selesai', 'batal') DEFAULT 'pending',
    status_bayar ENUM('belum_bayar', 'dp', 'lunas') DEFAULT 'belum_bayar',
    metode_bayar ENUM('cash', 'transfer') DEFAULT 'cash',
    catatan TEXT,
    created_by INT,
    updated_by INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_pesanan_pelanggan
        FOREIGN KEY (id_pelanggan) REFERENCES pelanggan(id_pelanggan)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Tabel `pesanan` merupakan header transaksi. Seluruh informasi utama order dicatat di sini, mulai dari pelanggan, jadwal kirim, status proses, status pembayaran, hingga total nilai transaksi.

### i. Tabel detail_pesanan

```
CREATE TABLE detail_pesanan (
    id_detail INT PRIMARY KEY AUTO_INCREMENT,
    id_pesanan INT NOT NULL,
    id_varian INT NOT NULL,
    qty INT NOT NULL,
    harga_satuan DECIMAL(10,2) NOT NULL,
    subtotal DECIMAL(15,2) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_detail_pesanan
        FOREIGN KEY (id_pesanan) REFERENCES pesanan(id_pesanan)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_detail_varian
        FOREIGN KEY (id_varian) REFERENCES varian_produk(id_varian)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Tabel `detail_pesanan` menyimpan rincian item pada setiap pesanan. Satu pesanan dapat memiliki banyak detail. Kolom `subtotal` mencatat hasil perkalian kuantitas dan harga satuan.

**j. Tabel pembayaran**

```sql
CREATE TABLE pembayaran (
    id_pembayaran INT PRIMARY KEY AUTO_INCREMENT,
    id_pesanan INT NOT NULL,
    jumlah DECIMAL(15,2) NOT NULL,
    tgl_bayar DATETIME NOT NULL,
    metode ENUM('cash', 'transfer') NOT NULL,
    bukti VARCHAR(255),
    keterangan TEXT,
    received_by INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_pembayaran_pesanan
        FOREIGN KEY (id_pesanan) REFERENCES pesanan(id_pesanan)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Tabel `pembayaran` mendukung transaksi bertahap, baik DP, cicilan, maupun pelunasan. Satu pesanan dapat memiliki lebih dari satu pembayaran. Kolom `received_by` mengaitkan transaksi dengan pengguna yang menerima pembayaran.

**k. Tabel pengiriman**

```sql
CREATE TABLE pengiriman (
    id_pengiriman INT PRIMARY KEY AUTO_INCREMENT,
    id_pesanan INT NOT NULL UNIQUE,
    tgl_kirim DATETIME,
    tgl_sampai DATETIME,
    status ENUM('menunggu', 'dalam_perjalanan', 'sampai', 'gagal') DEFAULT 'menunggu',
    driver VARCHAR(100),
    catatan TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_pengiriman_pesanan
        FOREIGN KEY (id_pesanan) REFERENCES pesanan(id_pesanan)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Tabel `pengiriman` menyimpan status distribusi pesanan. Satu pesanan hanya memiliki satu data pengiriman agar status pengiriman tetap konsisten.

### l. Tabel tracking_log

```sql
CREATE TABLE tracking_log (
    id_log INT PRIMARY KEY AUTO_INCREMENT,
    id_pesanan INT NOT NULL,
    status VARCHAR(50) NOT NULL,
    keterangan TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_tracking_pesanan
        FOREIGN KEY (id_pesanan) REFERENCES pesanan(id_pesanan)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Tabel `tracking_log` berfungsi sebagai riwayat perubahan status pesanan maupun status pembayaran. Data ini mendukung pelacakan proses dan audit transaksi.

### m. Tabel pengeluaran

```sql
CREATE TABLE pengeluaran (
    id_pengeluaran INT PRIMARY KEY AUTO_INCREMENT,
    tanggal DATE NOT NULL,
    kategori ENUM('bahan_baku', 'operasional', 'gaji', 'transportasi', 'lainnya') NOT NULL,
    deskripsi TEXT NOT NULL,
    jumlah DECIMAL(15,2) NOT NULL,
    created_by INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_pengeluaran_user
        FOREIGN KEY (created_by) REFERENCES pengguna(id_user)
        ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Tabel `pengeluaran` mencatat biaya operasional usaha. Pengelompokan kategori membantu penyusunan laporan keuangan yang lebih terstruktur.

### n. Tabel transaksi_kas

```sql
CREATE TABLE transaksi_kas (
    id_transaksi INT PRIMARY KEY AUTO_INCREMENT,
    tanggal DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    jenis ENUM('masuk', 'keluar') NOT NULL,
    sumber_tipe ENUM('pembayaran', 'pengeluaran', 'penyesuaian') NOT NULL,
    sumber_id INT,
    nominal DECIMAL(15,2) NOT NULL,
    keterangan TEXT,
    dibuat_oleh INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_transaksi_kas_user
        FOREIGN KEY (dibuat_oleh) REFERENCES pengguna(id_user)
        ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Tabel `transaksi_kas` merupakan buku kas terpusat untuk arus kas masuk dan keluar. Data pada tabel ini digunakan sebagai dasar perhitungan saldo kas.

**o. Tabel setting**

```
CREATE TABLE setting (
    id_setting INT PRIMARY KEY AUTO_INCREMENT,
    setting_key VARCHAR(100) NOT NULL UNIQUE,
    setting_value TEXT,
    deskripsi TEXT,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Tabel `setting` menyimpan konfigurasi aplikasi yang bersifat dinamis, seperti identitas toko, nomor kontak, kapasitas angkut, dan jam operasional.

**p. Relasi antar tabel**

Relasi antar tabel disusun menggunakan foreign key untuk menjaga integritas data. Relasi utamanya meliputi:

```
`produk.id_kategori` mengarah ke `kategori.id_kategori`
`varian_produk.id_produk` mengarah ke `produk.id_produk`
`pelanggan.id_zona` mengarah ke `zona.id_zona`
`pesanan.id_pelanggan` mengarah ke `pelanggan.id_pelanggan`
`detail_pesanan.id_pesanan` mengarah ke `pesanan.id_pesanan`
`detail_pesanan.id_varian` mengarah ke `varian_produk.id_varian`
`pembayaran.id_pesanan` mengarah ke `pesanan.id_pesanan`
`pengiriman.id_pesanan` mengarah ke `pesanan.id_pesanan`
`tracking_log.id_pesanan` mengarah ke `pesanan.id_pesanan`
`pengeluaran.created_by` mengarah ke `pengguna.id_user`
`transaksi_kas.dibuat_oleh` mengarah ke `pengguna.id_user`
```

Selain relasi utama tersebut, kolom audit pada beberapa tabel juga terhubung ke `pengguna` agar aktivitas operasional dapat ditelusuri dengan baik.

**q. Trigger otomatis**

Trigger digunakan untuk menjaga konsistensi data dan mengotomatisasi proses penting, yaitu:

1. `trg_before_insert_pesanan`
   Membuat nomor invoice otomatis berdasarkan tanggal dan urutan harian.
2. `trg_before_insert_detail`
   Menghitung `subtotal` saat detail pesanan ditambahkan.

3. `trg_after_insert_detail`

   Memperbarui `total_harga` dan `grand_total` setelah item pesanan bertambah.

4. `trg_after_delete_detail`

   Menyesuaikan kembali total pesanan ketika detail dihapus.

5. `trg_after_insert_pembayaran`

   Mencatat pembayaran ke `transaksi_kas` sebagai kas masuk dan memperbarui status bayar.

6. `trg_after_update_pesanan`

   Mencatat perubahan status pesanan dan status pembayaran ke `tracking_log`.

7. `trg_after_pesanan_selesai`

   Menambah total transaksi pelanggan dan menghitung piutang jika pesanan belum lunas.

8. `trg_after_insert_pesanan`

   Membuat data awal pengiriman dan log status awal saat pesanan dibuat.

9. `trg_after_insert_pengeluaran`

   Mencatat pengeluaran operasional ke `transaksi_kas` sebagai kas keluar.

**r. View laporan**

View disediakan untuk memudahkan penyajian data operasional dan keuangan tanpa harus menulis query kompleks berulang, yaitu:

1. `v_pesanan_lengkap`

   Ringkasan pesanan yang digabung dengan data pelanggan, zona, dan pengiriman.

2. `v_detail_pesanan_produk`

   Rincian item pesanan beserta produk, varian, kuantitas, dan subtotal.

3. `v_laporan_harian`

   Ringkasan penjualan harian yang memuat jumlah order, nilai penjualan, lunas, dan piutang.

4. `v_piutang`

   Daftar pelanggan dengan sisa hutang yang masih berjalan.

5. `v_katalog`

   Daftar produk aktif beserta kategori, varian, harga, dan stok.

6. `v_ringkasan_kas`

   Rekap total kas masuk, kas keluar, dan saldo akhir.

7. `v_laporan_kas_harian`

   Ringkasan pergerakan kas per tanggal.

**s. Storage procedure**

Stored procedure digunakan agar proses bisnis inti berjalan melalui alur yang baku, yaitu:

1. `sp_create_pesanan`

   Membuat pesanan baru dan menghitung ongkir berdasarkan zona pelanggan.

2. `sp_add_detail_pesanan`

   Menambahkan detail item pesanan dengan validasi minimum order.

3. `sp_update_status`

   Memperbarui status pesanan dan menyesuaikan status pengiriman.

4. `sp_bayar`

   Mencatat transaksi pembayaran, sedangkan pembaruan status bayar ditangani oleh trigger.

5. `sp_laporan_penjualan`

   Menghasilkan laporan penjualan berdasarkan rentang tanggal tertentu.

## VI. Hasil Query

### a. Tabel Pengguna

SELECT * FROM `pengguna`;

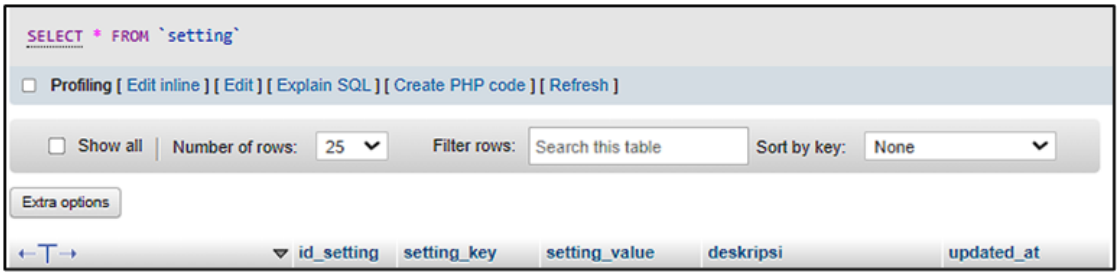
Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

| | id_user | username | password (Hashed password) | nama_lengkap | email | no_hp | role | status | last_login | created_at | updated_at |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Edit Copy Delete | 1 | admin | $2y$10$8K1p5s1VvqHqYL1YQs7xKOzR6h7L8P3n4Q5w6E7r8T9... | Administrator | NULL | NULL | admin | aktif | NULL | 2026-04-18 12:52:33 | 2026-04-18 12:52:33 |

### b. Tabel Setting



SELECT * FROM `setting`

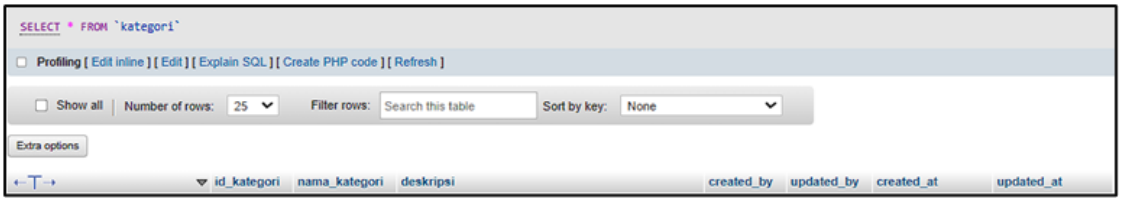
Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

| | id_setting | setting_key | setting_value | deskripsi | updated_at |
|---|---|---|---|---|---|

### c. Tabel Kategori



SELECT * FROM `kategori`

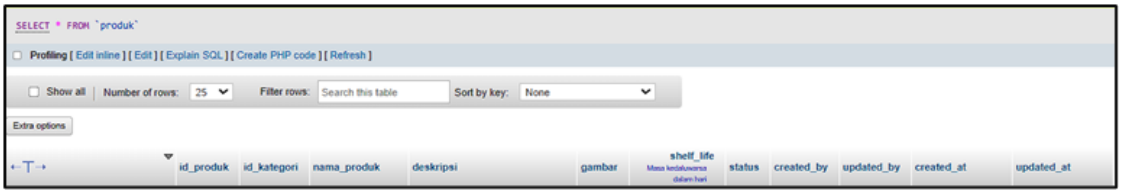
Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

| | id_kategori | nama_kategori | deskripsi | created_by | updated_by | created_at | updated_at |
|---|---|---|---|---|---|---|---|

### d. Tabel Produk



SELECT * FROM `produk`

Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

| | id_produk | id_kategori | nama_produk | deskripsi | gambar | shelf_life (Masa kedaluwarsa dalam hari) | status | created_by | updated_by | created_at | updated_at |
|---|---|---|---|---|---|---|---|---|---|---|---|

### e. Tabel Varian_produk

SELECT * FROM `varian_produk`

Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

| | id_varian | id_produk | nama_varian (Contoh: Kardus, Mika, Satuan) | harga | min_order (Minimal pembelian) | stok | created_at | updated_at |
|---|---|---|---|---|---|---|---|---|

### f. Tabel Zona

```
SELECT * FROM `zona`
```

Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

| ←T→ | id_zona | nama_zona | deskripsi | ongkir | created_at | updated_at |
|---|---|---|---|---|---|---|

## g. Tabel Pelanggan

```
SELECT * FROM `pelanggan`
```

Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

| id_pelanggan | id_zona | nama | alamat | no_wa | tipe | total_transaksi<br>Akumulasi total belanja | total_hutang<br>Total piutang belum lunas | catatan | created_at | updated_at |
|---|---|---|---|---|---|---|---|---|---|---|

## h. Tabel Pesanan

```
SELECT * FROM `pesanan`
```

Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

| id_pesanan | id_pelanggan | no_invoice | tgl_pesan | tgl_kirim<br>Deadline/jadwal pengiriman | waktu_kirim | total_harga | ongkir | grand_total | status | status_bayar | metode_bayar | catatan | created_by | updated_by | created_at | updated_at |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

## i. Tabel Detail_pesanan

```
SELECT * FROM `detail_pesanan`
```

Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

| id_detail | id_pesanan | id_varian | qty | harga_satuan | subtotal | created_at |
|---|---|---|---|---|---|---|

## j. Tabel Pembayaran

```
SELECT * FROM `pembayaran`
```

Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

| id_pembayaran | id_pesanan | jumlah | tgl_bayar | metode | bukti<br>Path file bukti transfer | keterangan | received_by | created_at |
|---|---|---|---|---|---|---|---|---|

## k. Tabel Pengiriman

```
SELECT * FROM `pengiriman`
```

Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

| id_pengiriman | id_pesanan | tgl_kirim | tgl_sampai | status | driver | catatan | created_at | updated_at |
|---|---|---|---|---|---|---|---|---|

## l. Tabel Tracking_log

```
SELECT * FROM `tracking_log`
```

Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create

| id_log | id_pesanan | status | keterangan | created_at |
|---|---|---|---|---|

## m. Tabel Pengeluaran

```
SELECT * FROM `pengeluaran`
```

Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

| id_pengeluaran | tanggal | kategori | deskripsi | jumlah | created_by | created_at |
|---|---|---|---|---|---|---|

## n. Tabel Transaksi_kas

```
SELECT * FROM `transaksi_kas`
```

Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

| id_transaksi | tanggal | jenis | sumber_tipe | sumber_id | nominal | keterangan | created_by | created_at |
|---|---|---|---|---|---|---|---|---|

## o. Stored Procedure sp_create_pesanan call query

Run SQL query/queries on database berkat_dinasti_db:

```
USE berkat_dinasti_db;

-- Input parameter
SET @p_id_pelanggan = 1;
SET @p_tgl_kirim = '2026-04-25';
SET @p_waktu_kirim = '09:00:00';
SET @p_metode_bayar = 'cash';
SET @p_catatan = 'Pesanan untuk acara';

-- Output parameter
SET @p_id_pesanan = 0;

CALL sp_create_pesanan(
    @p_id_pelanggan,
    @p_tgl_kirim,
    @p_waktu_kirim,
    @p_metode_bayar,
    @p_catatan,
    @p_id_pesanan
);

-- Tampilkan id pesanan yang baru dibuat (penting untuk prosedur berikutnya)
SELECT @p_id_pesanan AS id_pesanan_baru;
```

## p. Stored Procedure sp_add_detail_pesanan call query

Run SQL query/queries on database berkat_dinasti_db:

```sql
USE berkat_dinasti_db;

-- Gunakan id pesanan dari hasil sp_create_pesanan
SET @p_id_pesanan = 1;
SET @p_id_varian = 1;
SET @p_qty = 10;

CALL sp_add_detail_pesanan(
    @p_id_pesanan,
    @p_id_varian,
    @p_qty
);

-- Cek hasil detail yang masuk
SELECT *
FROM detail_pesanan
WHERE id_pesanan = @p_id_pesanan;
```

**q. Stored Procedure sp_update_status call query**

Run SQL query/queries on database berkat_dinasti_db:

```sql
USE berkat_dinasti_db;

SET @p_id_pesanan = 1;
SET @p_status = 'kirim';   -- opsi: pending, proses, kirim, selesai, batal

CALL sp_update_status(
    @p_id_pesanan,
    @p_status
);

-- Verifikasi status pesanan dan pengiriman
SELECT id_pesanan, status, status_bayar
FROM pesanan
WHERE id_pesanan = @p_id_pesanan;

SELECT id_pesanan, status, tgl_kirim, tgl_sampai
FROM pengiriman
WHERE id_pesanan = @p_id_pesanan;
```

**r. Stored Procedure sp_bayar call query**

Run SQL query/queries on database berkat_dinasti_db:

```sql
USE berkat_dinasti_db;

SET @p_id_pesanan = 1;
SET @p_jumlah = 50000.00;
SET @p_metode = 'cash';      -- opsi: cash, transfer
SET @p_bukti = NULL;         -- isi path jika transfer
SET @p_keterangan = 'Pembayaran tahap 1';
SET @p_received_by = 1;      -- id_user penerima pembayaran

CALL sp_bayar(
    @p_id_pesanan,
    @p_jumlah,
    @p_metode,
    @p_bukti,
    @p_keterangan,
    @p_received_by
);

-- Verifikasi pembayaran dan update status bayar
SELECT *
FROM pembayaran
WHERE id_pesanan = @p_id_pesanan
ORDER BY id_pembayaran DESC;

SELECT id_pesanan, grand_total, status_bayar
FROM pesanan
WHERE id_pesanan = @p_id_pesanan;
```

**s. Stored Procedure sp_bayar call query**

Run SQL query/queries on database berkat_dinasti_db:

```sql
USE berkat_dinasti_db;

SET @p_start_date = '2026-04-01';
SET @p_end_date = '2026-04-30';

CALL sp_laporan_penjualan(
    @p_start_date,
    @p_end_date
);
```

## VII. Kesimpulan

Pengembangan Sistem Manajemen Database Terintegrasi ini telah memberikan kesempatan berharga bagi mahasiswa untuk mengimplementasikan konsep basis data relasional secara komprehensif dalam lingkungan nyata. Melalui proyek kolaborasi dengan mitra Berkat Dinasti, mahasiswa tidak hanya berhasil menerapkan pengetahuan teoritis mengenai normalisasi data untuk menjaga integritas informasi, perancangan relasi antar-tabel yang kompleks, hingga penggunaan *trigger* dan *stored procedure* untuk otomatisasi bisnis, tetapi juga mengasah kemampuan dalam menganalisis kebutuhan operasional UMKM yang dinamis. Pengalaman ini memperkaya pemahaman bahwa sebuah struktur database yang solid merupakan fondasi utama dalam menciptakan sistem informasi yang transparan, akurat, dan efisien.

Secara lebih luas, proyek ini merefleksikan peran krusial akademisi dalam mendukung akselerasi transformasi digital UMKM di Indonesia. Implementasi sistem manajemen database yang terintegrasi mencakup pengelolaan produk, transaksi, pelanggan, hingga arus kas menawarkan solusi strategis bagi sektor UMKM untuk meninggalkan pencatatan manual yang rentan kesalahan. Digitalisasi ini memungkinkan pengambilan keputusan berbasis data yang lebih tajam, yang pada akhirnya dapat meningkatkan daya saing usaha di era ekonomi digital.

Dengan penyusunan *manual book* ini sebagai bagian dari tugas akhir dan pengajuan Hak Kekayaan Intelektual (HKI), diharapkan tercipta sebuah standar dokumentasi teknis yang aplikatif. Keberhasilan proyek ini membuktikan bahwa sinergi antara dunia pendidikan dan sektor industri mampu menghasilkan solusi teknologi yang tepat guna, berkelanjutan, serta berdampak nyata bagi penguatan fondasi ekonomi digital nasional dalam menghadapi tantangan industri 4.0.